Documentación del Proyecto #1 – AdoPaws: Plataforma Digital para Adopción, Marketplace y Consultas Profesionales para Mascotas

Albany Gonzalez.

José Pablo Monestel.

Avril Madrigal.

Rebeca Arce.

Universidad Invenio

Ingeniería de Software 1

Darin Mauricio Gamboa

25 de marzo de 2026

Resumen

El presente documento corresponde al Entregable 1 del Proyecto Final de la asignatura Ingeniería de Software 1. Su propósito es documentar formalmente los resultados obtenidos durante el desarrollo del Proof of Concept (PoC) de la plataforma AdoPaws, una solución digital orientada a facilitar la adopción responsable de mascotas, la comercialización de artículos relacionados con animales y la conexión entre usuarios y profesionales veterinarios. El informe analiza el cumplimiento de los 14 requisitos funcionales y siete reglas de negocio definidas para el PoC, identifica las lecciones aprendidas y los desafíos enfrentados durante las cuatro semanas de desarrollo, evalúa la viabilidad técnica demostrada por el stack seleccionado (React, ASP.NET Core, SQL Server) y emite recomendaciones orientadas al desarrollo del Producto Mínimo Viable (MVP).

Palabras clave: proof of concept, adopción de mascotas, desarrollo ágil, arquitectura SPA, viabilidad técnica, MVP.

Introducción

AdoPaws es una plataforma digital diseñada para atender tres necesidades principales en el ecosistema de mascotas: la adopción responsable, la compraventa de artículos especializados y la consulta con profesionales veterinarios verificados. El proyecto fue desarrollado en la asignatura Ingeniería de Software 1 de la Universidad Invenio, aplicando la metodología Scrum en ciclos de desarrollo de dos semanas.

El Proof of Concept (PoC) constituyó la primera fase del proyecto, con una duración de tres semanas. Su objetivo fue demostrar la viabilidad técnica del sistema mediante la implementación funcional de sus módulos esenciales en un entorno controlado. El stack tecnológico seleccionado comprende React 18 con Vite para el frontend, ASP.NET Core Web API con .NET para el backend, SQL Server como gestor de base de datos y Git/GitHub para el control de versiones.

El presente documento tiene como propósito consolidar los resultados de cada fase del PoC, analizar el grado de cumplimiento de los objetivos planteados, documentar las lecciones aprendidas y los principales desafíos enfrentados, evaluar la viabilidad técnica demostrada y formular recomendaciones concretas para el desarrollo del MVP.

Resultados Obtenidos en Cada Fase del PoC

El desarrollo del PoC se estructuró en cuatro fases consecutivas: análisis y definición de requisitos, diseño de arquitectura y base de datos, implementación y desarrollo, y validación y pruebas. A continuación, se presentan los resultados obtenidos en cada una de ellas.

Fase 1: Análisis y Definición de Requisitos

La primera fase se orientó al levantamiento y documentación de los fundamentos del sistema. Se definieron 14 requisitos funcionales (RF1–RF14), cubriendo el registro de usuarios, gestión de mascotas, publicación de artículos y sistema de consultas. Adicionalmente, se establecieron requisitos no funcionales relacionados con usabilidad, seguridad, rendimiento, integridad de datos, escalabilidad y mantenibilidad.

Se elaboraron siete reglas de negocio (RN1–RN7) que gobiernan el comportamiento del sistema, incluyendo las condiciones de registro, autenticación, publicación, gestión de solicitudes y control de acceso. Asimismo, se identificaron tres flujos críticos: autenticación de usuarios, operaciones CRUD de mascotas y gestión de solicitudes de adopción. El stack tecnológico fue seleccionado y justificado con base en los requerimientos del proyecto.

Los entregables de esta fase incluyeron el documento formal de alcance del PoC, el documento de reglas de negocio y flujos críticos, y la hoja de ruta de desarrollo para las cuatro semanas del proyecto.

Fase 2: Diseño de Arquitectura y Base de Datos

Durante la segunda fase se diseñaron los componentes estructurales del sistema. Se adoptó una arquitectura de tipo SPA más REST API, separando el frontend (React) del backend (ASP.NET Core). Para el backend, se aplicó Clean Architecture organizada en cuatro capas: Domain, Application, Infrastructure y API, garantizando que las dependencias apunten siempre hacia el dominio central.

Se modeló el Diagrama Entidad-Relación con seis entidades principales: Users, Pets, PetPhotos, AdoptionRequests, MarketplaceItems y Consultations/ConsultationResponses. La entidad PetPhotos fue diseñada como tabla independiente para facilitar la gestión de múltiples imágenes por mascota. Se definieron las relaciones y cardinalidades entre entidades, así como los índices únicos requeridos (correo electrónico y combinación IdPet/IdUser en solicitudes de adopción).

Tabla 1

Entidades del Modelo de Datos del PoC

Entidad	Relaciones Principales	Restricciones
Users	1:N con Pets, AdoptionRequests, MarketplaceItems, Consultations	Email UNIQUE
Pets	1:N con PetPhotos, AdoptionRequests	IdUser FK
PetPhotos	N:1 con Pets	IsMain flag

AdoptionRequests	N:1 con Pets y Users	UNIQUE (IdPet, IdUser)
MarketplaceItems	N:1 con Users	IdUser FK
Consultations / ConsultationResponses	1:N entre sí, N:1 con Users	SenderIdUser y ReceiverIdUser FK

Nota. Las entidades fueron implementadas en SQL Server mediante migraciones de Entity Framework Core.

Fase 3: Implementación y Desarrollo

La tercera fase concentró el mayor esfuerzo del PoC. Se implementaron cuatro módulos funcionales: Adopciones, Marketplace, Consultas y Gestión de Usuarios con Autenticación.

Módulo de Adopciones

Se desarrollaron los endpoints RESTful para el CRUD completo de mascotas, incluyendo soporte para múltiples fotografías con identificación de foto principal. El flujo de solicitudes de adopción fue implementado de extremo a extremo: envío, revisión y actualización de estado (Pendiente, Aceptada, Rechazada, Entrevista). En el frontend se construyeron las vistas de listado con filtros, detalle de mascota y formulario de solicitud.

Módulo de Marketplace

Se implementó el CRUD completo de artículos (MarketplaceItems) con los campos definidos en el modelo: Title, Category, Description, ItemCondition, Price, Region, MainPhoto y PublicationStatus. Se aplicó control de acceso para que únicamente el propietario de una publicación pudiera editarla o eliminarla.

Módulo de Consultas

Se desarrolló el sistema de mensajería entre usuarios y profesionales verificados, con entidades Consultations y ConsultationResponses. El frontend incorporó la visualización de perfiles de profesionales verificados, el formulario de envío de consultas y la bandeja de mensajes recibidos y enviados.

Módulo de Usuarios y Autenticación

Se implementó el registro de usuarios con validación de correo electrónico único y cifrado de contraseña, el inicio de sesión con generación de token JWT y la gestión de perfil con edición de datos personales. Se estableció la diferenciación de roles mediante el campo UserType y el atributo IsVerified para profesionales.

Fase 4: Validación y Pruebas

La fase final verificó el correcto funcionamiento de los flujos críticos identificados en el análisis. Se validaron los tres flujos prioritarios: autenticación (Login/Registro), operaciones CRUD de mascotas en sus cuatro variantes (Create, Read, Update, Delete) y el flujo completo de solicitudes de adopción desde el envío hasta la decisión del publicador.

Tabla 2

Estado de Validación de Flujos Críticos del PoC

Flujo Validado	Estado	Observaciones
Registro e inicio de sesión (JWT)	Completado	Validaciones activas, token generado correctamente
CRUD de mascotas – Create	Completado	Formulario con soporte de múltiples fotos
CRUD de mascotas – Read/Filter	Completado	Filtros por región, tipo y edad funcionales
CRUD de mascotas – Update	Completado	Control de propietario implementado
CRUD de mascotas – Delete	Completado	Eliminación funcional
Solicitud de adopción (flujo completo)	Completado	Envío, revisión y decisión operativos
CRUD de artículos (Marketplace)	Completado	Publicación y gestión funcionales
Envío y respuesta de consultas	Completado	Mensajería entre usuarios operativa
Gestión de perfil de usuario	Completado	Edición de datos y foto funcional

Nota. Todos los flujos críticos fueron validados exitosamente en el entorno de desarrollo.

Análisis de Cumplimiento de Objetivos

El PoC alcanzó un cumplimiento del 100% de los 14 requisitos funcionales y las siete reglas de negocio establecidas en el alcance del proyecto. La Tabla 3 presenta el estado de cumplimiento por requisito funcional.

Tabla 3

Cumplimiento de Requisitos Funcionales del PoC

ID	Requisito Funcional	Módulo	Estado
RF1	Registro de usuario	Usuarios	Cumplido
RF2	Inicio de sesión	Usuarios	Cumplido
RF3	Gestión de perfil de usuario	Usuarios	Cumplido
RF4	Publicación de mascotas	Adopciones	Cumplido
RF5	Visualización de mascotas	Adopciones	Cumplido
RF6	Búsqueda de mascotas	Adopciones	Cumplido
RF7	Envío de solicitud de adopción	Adopciones	Cumplido
RF8	Gestión de solicitudes de adopción	Adopciones	Cumplido
RF9	Publicación de artículos	Marketplace	Cumplido
RF10	Visualización de artículos	Marketplace	Cumplido
RF11	Gestión de artículos publicados	Marketplace	Cumplido

RF12	Visualización de perfiles profesionales	Consultas	Cumplido
RF13	Envío de consultas	Consultas	Cumplido
RF14	Respuesta de consultas	Consultas	Cumplido

Nota. 14 de 14 requisitos funcionales fueron implementados y validados durante el PoC (100% de cumplimiento).

Con respecto a las reglas de negocio, las siete reglas definidas (RN1–RN7) fueron implementadas satisfactoriamente. La regla RN3 referida al límite de cinco mascotas activas para usuarios con plan gratuito fue contemplada en el diseño del modelo de datos, aunque su lógica de restricción activa fue postergada al MVP por tratarse de una funcionalidad de monetización fuera del alcance del PoC.

El cumplimiento de los requisitos no funcionales fue satisfactorio en los aspectos de usabilidad, seguridad (contraseñas cifradas, autenticación JWT), integridad de datos (claves primarias, foráneas e índices únicos) y mantenibilidad (Clean Architecture). Los aspectos de rendimiento bajo carga real y escalabilidad de infraestructura serán evaluados en la fase del MVP.

Lecciones Aprendidas y Desafíos Enfrentados

Lecciones Aprendidas

Planificación y Gestión del Proyecto

La definición detallada del alcance antes de iniciar el desarrollo resultó determinante para mantener al equipo enfocado y evitar el scope creep. El documento de reglas de negocio funcionó como referencia permanente durante toda la fase de implementación, reduciendo la ambigüedad en la toma de decisiones técnicas. La aplicación de Scrum con ceremonias semanales de revisión facilitó la identificación temprana de desviaciones y su corrección oportuna.

La priorización explícita de los flujos críticos (autenticación, CRUD de mascotas y gestión de solicitudes de adopción) garantizó que las funcionalidades de mayor valor estuvieran operativas antes del cierre del PoC, permitiendo destinar el tiempo restante a módulos secundarios.

Arquitectura y Decisiones Técnicas

La adopción de Clean Architecture para el backend demostró ser una decisión acertada: la separación entre Domain, Application, Infrastructure y API facilitó el desarrollo paralelo de capas y estableció una base sólida para la incorporación de nuevas funcionalidades en el MVP. El uso de Entity Framework Core con migraciones simplificó la gestión del esquema de base de datos,

reduciendo el tiempo dedicado a scripts SQL manuales y garantizando la consistencia del modelo a lo largo del desarrollo.

La implementación de JWT para autenticación desde la primera fase del PoC sentó las bases correctas de seguridad y resultó compatible con los requerimientos de control de acceso por roles que el MVP deberá implementar de forma más granular.

Desarrollo Frontend

React como SPA permitió construir una interfaz dinámica y fluida, particularmente en los flujos de búsqueda y filtrado de mascotas. La gestión del estado global de autenticación mediante Context API resultó suficiente para el alcance del PoC, aunque el MVP podría beneficiarse de una solución más robusta como Redux Toolkit o Zustand para manejar la complejidad adicional de los nuevos módulos.

Desafíos Enfrentados

Desafíos Técnicos

La configuración inicial de CORS entre el frontend (React en puerto local) y el backend (ASP.NET Core) requirió ajustes específicos en el pipeline de middleware para permitir las solicitudes cross-origin durante el desarrollo. Este desafío fue resuelto de forma centralizada en Program.cs, aunque la configuración deberá ser restringida para entornos productivos en el MVP.

El manejo de la carga y almacenamiento de imágenes para las fotos de mascotas y artículos del marketplace representó uno de los principales desafíos técnicos del PoC. La solución adoptada durante esta fase (URLs externas) es funcional para el entorno de pruebas, pero no es escalable para el MVP, donde se requerirá una estrategia de almacenamiento persistente en la nube.

La implementación del sistema de roles y control de acceso implicó lógica adicional en los endpoints del backend para validar permisos por operación, asegurando que únicamente el propietario de un recurso pudiera editarlo o eliminarlo. Esta validación fue aplicada de manera consistente en los módulos de Adopciones, Marketplace y Consultas.

Desafíos de Gestión del Equipo

La coordinación del trabajo paralelo en frontend y backend exigió mantener consistencia en los contratos de los endpoints, incluyendo nombres de campos, tipos de datos y códigos de respuesta HTTP. Este desafío fue mitigado mediante la documentación automática de Swagger, que actuó como fuente de verdad compartida para ambas capas.

El equilibrio entre la velocidad de desarrollo requerida por el PoC y la calidad del código fue una tensión constante. La decisión del equipo fue priorizar la funcionalidad completa sobre las optimizaciones avanzadas, lo cual es coherente con los objetivos de la fase. Sin embargo, este enfoque generó deuda técnica que deberá ser atendida durante el MVP mediante la implementación de pruebas unitarias y análisis de calidad con SonarQube.

Conclusiones sobre la Viabilidad Técnica Demostrada

El PoC de AdoPaws demostró de manera sólida y documentada la viabilidad técnica de la plataforma. Las conclusiones se presentan a continuación en función de los ejes de evaluación más relevantes.

Viabilidad del Stack Tecnológico

La combinación React + ASP.NET Core Web API + SQL Server demostró ser técnicamente coherente y productiva para el contexto del proyecto. La integración entre las tres capas funcionó sin impedimentos críticos durante las cuatro semanas de desarrollo, y el stack permitió alcanzar la implementación completa del alcance definido dentro del plazo establecido. Esta combinación tecnológica es madura, cuenta con amplia documentación y comunidades activas, lo que reduce el riesgo técnico para la fase del MVP.

Viabilidad del Modelo de Datos

El DER implementado demostró ser suficientemente flexible para soportar los tres módulos del sistema. Las relaciones entre entidades se comportaron correctamente bajo los flujos de prueba, incluyendo los casos de uso más complejos como el control de duplicados en solicitudes de adopción (restricción única sobre la combinación IdPet/IdUser) y la gestión de múltiples fotografías con identificación de imagen principal. La estructura de datos está preparada para escalar hacia el MVP sin requerir refactorizaciones mayores del esquema.

Viabilidad de la Arquitectura SPA + API REST

La arquitectura de aplicación de página única en el frontend, consumiendo una API RESTful en el backend, demostró ser adecuada para los casos de uso de AdoPaws. Los tiempos de respuesta fueron satisfactorios para las operaciones CRUD y de búsqueda en el entorno de desarrollo. La separación de responsabilidades entre frontend y backend facilita el mantenimiento independiente de cada capa y permite incorporar nuevos desarrolladores sin acoplamiento excesivo entre componentes.

Viabilidad Funcional

Los flujos más críticos del sistema operaron correctamente en el entorno de pruebas: autenticación con JWT, publicación y búsqueda de mascotas con filtros múltiples, y el ciclo completo de solicitudes de adopción desde el envío hasta la resolución por parte del publicador. Este resultado valida que la lógica de negocio definida en las reglas de negocio es implementable con el stack seleccionado y dentro de los plazos del proyecto.

En síntesis, el PoC de AdoPaws cumplió con el 100% de sus objetivos funcionales y técnicos, estableciendo una base arquitectónica sólida y demostrando la capacidad del equipo para desarrollar un producto de software funcional bajo la metodología Scrum. El proyecto se encuentra en condiciones óptimas para avanzar hacia el desarrollo del MVP.

Recomendaciones para el Desarrollo del MVP

Con base en los resultados del PoC, los desafíos identificados y el análisis de viabilidad técnica, se presentan las siguientes recomendaciones organizadas por área de intervención.

Arquitectura e Infraestructura

Se recomienda implementar una solución de almacenamiento de archivos en la nube (Azure Blob Storage o Amazon S3) para la gestión de imágenes de mascotas y artículos del marketplace. La estrategia de URLs externas utilizada en el PoC no es viable para un entorno productivo con usuarios reales. Esta implementación debe ser prioritaria en el primer sprint del MVP, dado que impacta directamente en la funcionalidad de los módulos de Adopciones y Marketplace.

Se sugiere configurar entornos separados de desarrollo, staging y producción desde el inicio del MVP, con variables de entorno específicas por ambiente. Esto reducirá el riesgo de errores de configuración durante el proceso de CI/CD y facilitará las pruebas de integración.

Calidad y Pruebas

El enunciado del MVP establece una cobertura mínima de pruebas unitarias del 70% en módulos críticos. Se recomienda iniciar la escritura de tests desde el primer sprint, priorizando los servicios de mayor complejidad lógica: UserService (validaciones de registro), PetService (CRUD

con control de propietario) y AdoptionRequestService (flujo de estados). Postergar la implementación de pruebas al final del proyecto aumenta significativamente el costo de corrección y reduce la cobertura alcanzable en el tiempo disponible.

La integración de SonarQube desde el inicio del MVP, y no al final, permite detectar code smells y deuda técnica de forma continua. Se recomienda configurar los Quality Gates definidos en el enunciado (cobertura ≥ 70%, duplicación < 3%, Mantenibilidad A o B, Confiabilidad A o B, Seguridad A, 0 bugs críticos) como condición de aprobación del pipeline desde el primer workflow de GitHub Actions.

CI/CD y Prácticas DevOps

Se recomienda configurar el pipeline de GitHub Actions desde la primera semana del MVP, implementando los cinco stages requeridos: Build, Test, SonarQube Analysis, Security Check y Deploy Staging simulado. La detección temprana de errores de integración reduce el costo de corrección y evita la acumulación de conflictos en sprints avanzados. Se sugiere aplicar una estrategia de branching basada en GitHub Flow, con Pull Requests que requieran al menos un revisor antes de hacer merge a la rama principal.

Funcionalidades Prioritarias

En cuanto a la expansión funcional requerida por el MVP, se recomienda priorizar los siguientes módulos adicionales al PoC: (a) sistema de recomendaciones con motor de

compatibilidad usuario-mascota, que entrega valor diferencial a los adoptantes; (b) módulo de favoritos para que los usuarios guarden mascotas de interés; (c) panel de gestión (dashboard) para cuentas de tipo refugio o asociación, que centraliza la administración de publicaciones y solicitudes; y (d) paginación del lado del servidor en los listados de mascotas y artículos del marketplace, necesaria para garantizar el rendimiento con volúmenes reales de datos.

Adicionalmente, se sugiere implementar validaciones robustas en los DTOs del backend mediante FluentValidation o Data Annotations, dado que el PoC priorizó la funcionalidad sobre la validación exhaustiva de entradas. Esta mejora reducirá la superficie de errores y contribuirá a los ratings de calidad en SonarQube.

Seguridad

Se recomienda implementar refresh tokens en el sistema de autenticación JWT para mejorar la experiencia del usuario sin comprometer la seguridad de las sesiones. Adicionalmente, se sugiere agregar validación del tipo y tamaño de archivos en los endpoints de carga de imágenes, así como un mecanismo de verificación de correo electrónico durante el registro para reducir la creación de cuentas fraudulentas.

Referencias

- *Humble, J., & Farley, D. (2010). Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.
- Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.
- Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.
- Microsoft. (2024). *ASP.NET Core documentation*. https://learn.microsoft.com/en-us/aspnet/core/
- Meta Open Source. (2024). *React documentation*. https://react.dev/
- Schwaber, K., & Sutherland, J. (2020). *The Scrum guide: The definitive guide to Scrum – The rules of the game*. Scrum.org. https://scrumguides.org/
- SonarSource. (2024). *SonarQube documentation*. https://docs.sonarsource.com/sonarqube/